

Weekly Update Documentation

By: Ghanashyam R K P <ghanashyamrkp@gmail.com>

Week 1: 07.01 - 07.06 (Working Days of the week : Thursday - Monday)

This week I planned to learn how to execute the cartesian path following (Point to Point) using Moveit! on the kuka robot.

Tasks done this week :

- Revisited working with ROS, Moveit! and Gazebo using [Mastering ROS for Robotics Programming - Second Edition Free eBook](#). It helped me correct some of my misconceptions that I had earlier.
- Completed ROS documentation -  ROS Doc - Outline.pdf .
 1. Lib_Py
 2. Lib_CPP
- Created ROS Package to execute move_group_python_tutorials on the kuka robot.
- Spent some time exploring plugins / packages which already implements cartesian paths.
- Found one - [moveit cartesian plan plugin - ROS Wiki](#)
 1. Compiled it for ROS noetic.
 2. It provides a widget interfaced as a panel with Rviz, which can be used to load cartesian waypoints through a YAML file, then the plugin helps execute the plan.
 3. Tried with some sample points. Have to tweak it for the trajectory points generated by Abhijay.
- Also tried with my code for executing cartesian points by the help of moveit tutorials.

Plan for next week :

- Go through [Pilz Industrial Motion Planner — moveit tutorials Noetic documentation](#) and learn how to use it for the trajectory points of the seam
- Go through the ROS Action Library in a detailed manner.

Weekly Update Documentation

By: Ghanashyam R K P <ghanashyamrpk@gmail.com>

Week 2: 07.07 - 07.12

Tasks Planned for Week 2:

- Configure and use Pilz Industrial Motion Planner.
- Go through the ROS Action Library in detail (Ongoing task).
- Go through Quaternions, Euler angles.

Challenges this week :

- Pilz industrial motion planner was very recently moved to moveit! as one of its planners, which lacks proper docs on integration, had to figure out on how to compile them in the first place.
- So the moveit! config package had to be updated (Which I didn't realise at first, because of not encountering this before), then I did it using `setup_assistant` to generate files which will help the planner to work.
- Then I found the `pilz_industrial_motion_planner` library here, which I downloaded and compiled.
https://github.com/ros-planning/moveit/tree/master/moveit_planners/pilz_industrial_motion_planner
- For learning about Quaternions I followed this wonderful video
 Visualizing quaternions (4d numbers) with stereographic projection

Challenges / Plans for next week :

- Learn how to use `pilz_industrial_motion_planner`.
- As a backup option - configure and use `descartes` plugin for path planning.
- Continue learning ROS Action Library.

Any more challenges / Plans will be added after tomorrow's meeting.

Weekly Update Documentation

By: Ghanashyam R K P <ghanashyamrpk@gmail.com>

Week 3: 07.13 - 07.20

Tasks Planned for Week 3:

- Using a Planner to traverse the points given by abhijay.
- Continue learning ROS Action Library.
- Explore ways of Rotation of the auger and the configuration of the weld bit w.r.t auger.

Challenges and Tasks Done:

- Executed the path by converting the points given by abhijay (of an upright standing auger and Horizontally oriented auger), into a yaml file of the required format of Moveit_Cartesian_Path_Planner.
 - Scaled down the points to make the arm possible to reach the points
 - even though this is not exactly what we'll do as we are going to rotate the point cloud and move the arm linearly, this I did so as to make sure
 - Planner does the job properly with the yaml file created.
 - To take care of the scaling
 - To tweak the orientation of the points to make it horizontal w.r.t to the arm.
- Have decided to go with this planner currently.
- Continued with ROS Actions library and implemented a small task in the course in order to grasp the concepts more clearly.
- Looked into the tf2 package in-order to understand it better.
- For Rotation of Point cloud
 - Pcl Transform Library - It can be used for rotation by giving proper configurations / values of the angles.

Tasks / Challenges for Next Week:

- Rotate the point cloud with Pcl Transform Library, and also look ways for configuring the motion of the end-effector of the arm w.r.t the auger.

Weekly Update Documentation

By: Ghanashyam R K P <ghanashyamrkp@gmail.com>

- Complete the capstone of the ROS Actions Library Course

Week 4: 07.20 - 07.27

Tasks Planned for Week 3:

- Rotate the Point Cloud using PCL Transform.
- Look for ways to configure end-effector
- ROS Action Library Course

Challenges and Tasks Done:

- Used Point Cloud Library to rotate a .pcd (Point Cloud Data File) - using both rotation matrix and Eigen transform functions.
 - Takes the angle of rotation as input.
 - If translation of Point Cloud is needed that can also be done.
- Created a Custom C++ library of the same, for generic usage.
- Wrote the code for a node in ROS using C++ which
 - Subscribes the scanned point cloud data.
 - Rotates it to a specific angle, using the library “rotate_pc” created in the previous step.
 - Publishes it to another topic / it can be saved to a file so that when the rest of the point cloud is scanned after rotation, both can be concatenated to get the complete point cloud.
- ROS Action Library Course - Ongoing (will complete it by next week)

Challenges and Task for Next Week:

- Integrate the Rotation Code with abhijay’s code.
- Accordingly start looking into configuring the end-effector
- Completing ROS Action Library Course.

Weekly Update Documentation

By: Ghanashyam R K P <ghanashyamrkp@gmail.com>

Break* : 07.28 - 08.02

**due to important home university work (Completed ROS Actions Library Course)*

Week 5 : 08.03 - 07.09

Tasks Planned for Week 3:

- Configuring the movement of the end-effector and the rotation of the point cloud (Plan and Math)

Challenges and Tasks Done:

- Formulated a pipeline for the rotation of point cloud and movement of the end-effector.
- Using Point-cloud of the auger (Ideal) - derived **linear consecutive positions** of the gripper.
- Using Point-cloud of the auger (Ideal) - derived the **angles** that the point-cloud should rotate in order for the end-effector to follow and correctly weld at the specific points on the auger. (I used limited points, which is not efficient, but we can customize to use any number of points we want).
 - Used basic inverse trigonometric (specifically arctan2 (tan inverse) function of the numpy library) to derive angles.
- Interesting thing about this is these calculations would work on any auger (not-limited to the ideal one). Nothing is hard-coded.

Challenges and Task for Next Week:

- Configuring the movement of the end-effector and rotation of the point cloud (Implementing it in simulation).

Weekly Update Documentation

By: Ghanashyam R K P <ghanashyamrkp@gmail.com>

Week 6 : 08.10 - 08.16

Tasks Planned for Week 6:

- Configuring the movement of the end-effector and rotation of the point cloud (Implementing it in simulation).

Challenges and Tasks Done:

- Made corrections to the last week's approach
- New Approach
 - Initially Rotating the auger in such a way that, angle made by the first point with the vertical axis = 0.
 - Then the end-effector will be moved to that position and welding starts.
 - After this iteratively till the end of the coordinates of the seam
 - subscribe to the previous rotated point cloud, Rotate it w.r.t vertical axis, publish it again to a topic, which shall be subscribed by the main node and rendered in simulation.
 - Now move from the previous coordinates to the new coordinates.
- Another approach that I would like to experiment with is, without rotating, traversing the points till the auger no more can do it due to limits, then only rotate, this will be more efficient than the first approach.
- Calculated angles w.r.t vertical axis. Modifications in existing ros nodes and creation of new ones required for the process.

Challenges and Task for Next Week:

- Writing ROS nodes to implement all the above points accordingly.

Weekly Update Documentation

By: Ghanashyam R K P <ghanashyamrpk@gmail.com>

Week 7 : 08.17 - 08.23

Tasks Planned for Week 7:

- Writing ROS nodes to implement all the above points accordingly.

Challenges and Tasks Done:

- Rotation of point-cloud done using axis-angle method. Reusing the code I already wrote for scanning the point-cloud.
- Node written to get point-cloud and rotate it, move from current pose to next pose.
- Read about Descartes cartesian planner and usage. Implemented it's tutorials.

Challenges for Next Week:

- The node I have currently written takes a point-cloud published, rotates it, then moves the end-eff from previous pose to new pose.
- Use Descartes Planner in the above code for cartesian planning.
- Next step is to verify if this works after integrating with abhijay's code.
- If successful I will write a service / action to complete the sequence of steps for welding till the end.

Weekly Update Documentation

By: Ghanashyam R K P <ghanashyamrpk@gmail.com>

Week 8 : 08.24 - 08.30

Considering the feedback given, I focused more on calculating “z” and the “theta” of the whole process in one-go before the actual welding process. Accordingly, tasks were modified. All tasks were experimented using the pointcloud of an ideal-auger / model created by abhijay.

Tasks Planned for Week 8:

- Calculation of “Z” value and the angles to rotate the point-cloud.

Challenges and Tasks Done:

- Calculated the “Z” value of the whole path. That is the difference b/w each points in the point cloud.
- Wrote a ROS node to publish the point-cloud of seam to a topic.
- Wrote a ROS node that would subscribe to a point-cloud and calculate the angle.
- Defined a pipeline to continuously rotate the seam until the end of the points in the point-cloud to get all the angles required.
 - In this All nodes have been defined.
 - Created a custom msg file - (PointCloud2 and the angle in radians). This custom msg will be passed on to perform rotation every time.
 - A slight modification to pass this as the message to the previously written “point-cloud rotate” node.
 - Testing of the code remains.

Tasks for Next Week:

- Completing the testing. After the test, abhijay’s code will be used to get the point-cloud directly in ROS.
- This will be then used to calculate the angles.
- Post which a pipeline for the actual welding process and coding will be started.

Weekly Update Documentation

By: Ghanashyam R K P <ghanashyamrkp@gmail.com>

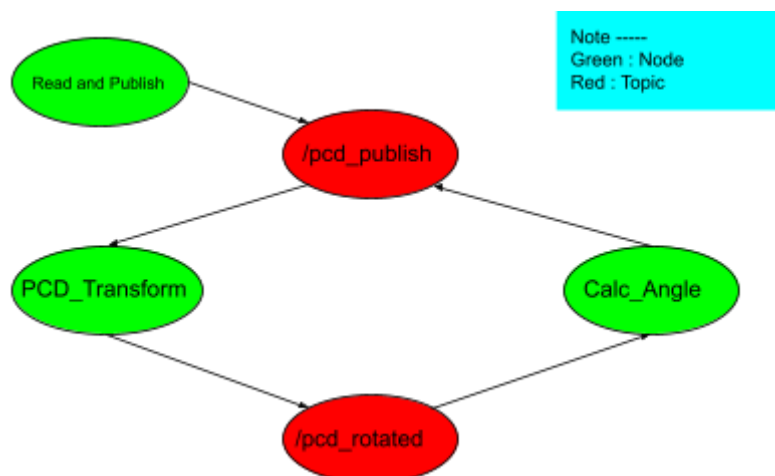
Week 9 : 08.31 - 09.06

Tasks Planned for Week 8:

- Completing, debugging and generating angles of rotation.

Challenges and Tasks Done:

- Completed Coding the pipeline with Three Nodes.
 - **Read_and_Publish** (This will publish the initial value of the point cloud data, along with initial angle of rotation) - For Once.
 - **PCD_Transform** Node will subscribe to the (/publish_pcd) topic and get the initial values of pcd and theta, and rotate it and publish the new pcd and theta to (/pcd_rotated). [In Loop]
 - **Calculate_Angle** Node will subscribe to the (/pcd_rotated) topic get the rotated pcd and theta, then
 - It will calculate the next angle
 - Save it to a file
 - along with pcd and **new** angle publish the message to (/pcd_publish) topic.
 - This process is repeatedly done in a loop to generate and save all angles.
- Debugging in process.



Weekly Update Documentation

By: Ghanashyam R K P <ghanashyamrkp@gmail.com>

Tasks for Next Week:

- Debug Abhijay's code.
- Then Define and start coding the pipeline for the welding process.

Week 10 : 08.07 - 09.17

Tasks Planned for Week 10:

- Getting Angles, and Experiment with cartesian path planning.

Challenges and Tasks Done:

- The angles that were generated last friday, is not exactly correct, and I realised it after converting them to degrees to use them
- It was wrong not due to the logic used, but due to the node structure itself, as in the rotation node is taking time to publish the new configuration of the points, but before that itself, the calc_angle node is calculating the angles, which is wrong.
- I experimented with ros::Rate and there is no improvement on that part.
- So I have decided to implement the node as just one single node for angle calculation, and I would be using the Open3D library for it, as previously the node written for rotation is in C++ (Which is perfectly correct). But I have to use python as I am using numpy.
- I tried with the normal cartesian path planning methods of moveit and I observed that they didn't work as expected. (Like it won't calculate the path after reaching to a pose).
- I interfaced Pilz Planner with our package only (It works fine in rviz GUI). I have to write code and test, that's remaining.

Tasks for Next Week:

- Generate the angles. (Using Open3D Rotation)
- Use pilz planner and achieve motion planning by writing code [This is one part of the final pipeline]

Weekly Update Documentation

By: Ghanashyam R K P <ghanashyamrkp@gmail.com>

- After angles generated successfully, I would have to visualize rotation in Rviz.

Week 11 : 09.18 - 09.23

Tasks Planned for Week 10:

- Cartesian Path Planning and Generating angles with Open3D

Challenges and Tasks Done:

- Used the “Z” value of the point cloud to execute a linear path for welding.
 - In Rviz and Gazebo - Stored the data as a “.npz” file after generation and used it for waypoints generation using OMPL planner.
 - Then first, the arm is moved to the starting point of the auger.
 - Subsequently, the arm is made to traverse all the points linearly, using the default computeCartesianPath method of moveit.
 - [NOTE : Currently the arm’s orientation is a bit different, but if the auger is planted to the planning scene, the orientation of the arm would become correct as it will take into account the collision object and plan accordingly]
- I generated the angles using Open3D in an ipython notebook and now the code is proper, but when I plotted the point clouds, it seems the logic in giving the individual angles to the axis-angle method is wrong. (So I have to do some corrections and see the changes.)

Tasks for Final Week:

- Correction in angle calculation.
- Simulating both the tasks in the gazebo together.

Weekly Update Documentation

By: Ghanashyam R K P <ghanashyamrpk@gmail.com>

Week 12 : 09.24 - 09.30

Tasks Planned for Week 10:

- Correction in angle calculation.
- Simulating both the tasks in the gazebo together.

Challenges and Tasks Done:

- Corrected the angle generation algorithm.
 - Algorithm is generalised for any auger whose base circle's center is Origin.
 - One more level of generalisation can be done where the circle's center is any (x,y).
- Saved the angles as an .npy file
- Coded a node in ROS and completed the simulation, which
 - Takes the angles array and Intersection-PointCloud as input
 - At each step the Point Cloud is rotated and the end-effector of the arm is made to point and weld in the new pose.
 - Rotation is performed using Open-3D.

Final Tasks:

- Document everything done till today.
- Formulate one more level of generalisation of angle generation code.

Final Tasks completed.

- Documented everything.
- Generalisation is already available after thorough analysis. The angle generation works for any point-cloud of the welding path. Only thing to make sure is the axes (which has been taken care in the

Weekly Update Documentation

By: Ghanashyam R K P <ghanashyamrkp@gmail.com>
